Nice HMS ABDM APIs — Error Reporting Reference

> ***Audience:*** *external integrators consuming the Nice HMS ABDM data-sharing APIs. This document is the authoritative reference for* ***how errors are returned*** *so you can refine your error handling and advance your integration. It is not an implementation guide.*

---

1. Base URL

All endpoints are called with an `Authorization: Bearer <idToken>` header (except `/auth_token`, which issues that token). Endpoint slugs are listed in §7.

```
https://asia-south1-psychic-city-328609.cloudfunctions.net/<endpoint>
```

- Default method is **POST**.
- `/discharge_patient` is **PUT**.
- `/health_document` is **multipart/form-data** (POST).

---

2. Error envelope (every non-2xx response)

Every error response uses the same JSON shape. There are no other error shapes.

```
{
  "message": "Human-readable summary of the failure",
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid request body",
    "details": [
      { "field": "patient.abhaAddress", "message": "The direct input of Abha Address or Abha number is not possible." }
    ]
  }
}
```

Rules

Field	Always present?	Meaning
`message`	Yes	Top-level summary (kept for backward compatibility — safe to keep parsing it).
`error.code`	Yes	Stable, machine-readable code. Match on this. See §4.
`error.message`	Yes	Curated, human-readable message.
`error.details`	**No** — only on `VALIDATION_ERROR`	Array of `{ field, message }`, one entry per failed field.

Example — non-validation error

```
{
  "message": "Patient not found",
  "error": {
    "code": "PATIENT_NOT_FOUND",
    "message": "Patient not found"
  }
}
```

Example — validation error

```
{
  "message": "Invalid request body",
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid request body",
    "details": [
      { "field": "patient.mobile", "message": "Invalid mobile number" },
      { "field": "patient.dob", "message": "Date of birth must be a valid past date" }
    ]
  }
}
```

---

3. Response headers

Every response — success or error — includes the same headers.

`x-request-id`

- If your request included an `x-request-id` header, its value is **echoed back** unchanged.
- Otherwise a new UUID is generated.
- The value lives in the **header only** — never inside the JSON body.

Send an `x-request-id` on every request and log the request/response pair against it. When escalating an issue to Nice HMS support, include this value — it correlates to the server-side log of the full (unredacted) error.

`Access-Control-Allow-Origin`

Every response includes `Access-Control-Allow-Origin: *`, so the API can be called directly from browser-based clients (CORS is enabled for all origins).

---

4. Error code taxonomy

Match on `error.code` — these are stable strings.

`error.code`	HTTP	Meaning
`VALIDATION_ERROR`	400	Request body failed schema validation. See `error.details` for field-level reasons.
`INVALID_EMAIL_FORMAT`	400	An email string is malformed.

`INVALID_DOCTOR_GCP_FHIR_ID`	400	A `doctorGcpId` does not exist in the FHIR store (Practitioner).
`DOCTOR_DETAILS_REQUIRED`	400	`doctorDetails` is missing (health_document).
`INVALID_REQUEST_METHOD`	400	Wrong HTTP method for the endpoint.
`UNAUTHORIZED`	401	Missing/invalid/expired token, or the token is not mapped to an organization.
`SUBSCRIPTION_EXPIRED`	403	The organization's subscription is invalid or expired.
`PATIENT_NOT_FOUND`	404	No patient matches the supplied `patientId` / `abhaAddress` for this organization.
`ENCOUNTER_NOT_FOUND`	404	No open encounter, or the supplied `encounterId` was not found.
`DOCTOR_NOT_FOUND`	404	The Practitioner FHIR id does not exist (edit_doctor).
`EMAIL_ALREADY_EXISTS`	409	Duplicate email when creating a doctor.
`ABDM_GATEWAY_ERROR`	502	Upstream ABDM gateway returned an error (e.g. OTP verification failed, auth mode unavailable).
`LINK_TOKEN_MISSING`	502	ABDM care-context link token was not found after the callback.
`DB_ERROR`	500	A database operation failed. Curated generic message.
`INTERNAL_ERROR`	500	Any unhandled / unknown error. Curated generic message.

---

5. HTTP status code summary

Status	Category
200	Success
400	Bad request / validation
401	Unauthorized
403	Forbidden (subscription)
404	Not found
409	Conflict (duplicate)
500	Internal server error
502	Bad gateway (ABDM upstream)

---

6. Validation semantics (important)

1. **Extra / unknown fields are ignored (stripped), not rejected.** You will not get an error for sending fields the API doesn't know about.
2. **Validation runs before auth.** A request that is both unauthenticated *and* has an invalid body will return `400 VALIDATION_ERROR` first (auth is checked after body validation). Plan your client accordingly.

3. **Field paths** in `details[].field` use dot notation, e.g. `patient.firstName`, `doctorDetails.0.doctorGcpId`, `patient.dob`.
4. **Curated messages only.** Responses never leak stack traces, SQL, DB error text, GCP FHIR paths/IDs, or raw ABDM payloads. The full unredacted cause is logged server-side against the `x-request-id`.

---

7. Per-route error reference

"Shared codes" (present on every **authenticated** route) are:

> *`VALIDATION_ERROR` (400)* · *`UNAUTHORIZED` (401)* · *`SUBSCRIPTION_EXPIRED` (403)* · *`DB_ERROR` (500)* · *`INTERNAL_ERROR` (500)*

Each route below lists the **shared codes + its own additional codes**.

Where a route says **"patientNeeds"**, the body must include at least one of `abhaAddress` (string) or `patientId` (number). Omitting both yields `400 VALIDATION_ERROR` with the message "*Either abhaAddress or patientId is required*".

7.1 `/auth_token` — POST (public, no token)

Issues the Firebase ID token used to authenticate every other endpoint.

Request fields

- `email` — valid email (required)
- `password` — non-empty (required)
- `returnSecureToken` — optional boolean

Error codes

- `VALIDATION_ERROR` (400) — malformed body
- `UNAUTHORIZED` (401) — invalid email or password
- `DB_ERROR` (500) · `INTERNAL_ERROR` (500)

Notes: Not authenticated; does not check subscription.

7.2 `/create_patient` — POST (auth)

Creates a **non-ABHA** patient (MySQL + GCP FHIR).

Request fields — `patient` object

- `firstName` — required, non-empty
- `gender` — `"Male" | "Female" | "Other"`
- `dob` — `YYYY-MM-DD`, a valid **past** date
- `mobile` — exactly 10 digits
- `state` — required, non-empty
- `district` — required, non-empty
- `abhaAddress` / `abhaNumber` — **rejected** (see note)
- optional passthrough: `lastName`, `middleName`, `prefix`, `address`, `email`, `pincode`, `pan`, `adhar`, `phone`, `internalid`, etc.

Error codes

- Shared codes only (no route-specific extras).

Notes: ABHA patients must be created via `/patient_register_abha_otp_init` + `/patient_register_abha_otp_confirm`. Passing `abhaAddress` or `abhaNumber` here returns `400 VALIDATION_ERROR` with message "*The direct input of Abha Address or Abha number is not possible.*"

7.3 `/create_doctor` — POST (auth)

Creates a doctor (Practitioner in FHIR + MySQL user).

Request fields

- `user.email` — valid email
- `user.firstName`, `user.lastName` — required
- `user.mobile` — 10 digits
- `doctorPrintName`, `medicalLicenseNumber`, `specialty`, `qualification` — required
- `gender` — `"Male" | "Female" | "Other"`
- `abdmProfessionalId` — optional

Error codes

- `VALIDATION_ERROR` (400)
- `INVALID_EMAIL_FORMAT` (400) — backstop for malformed email
- `EMAIL_ALREADY_EXISTS` (409) — email already registered
- shared codes

7.4 `/edit_doctor` — POST (auth)

Updates an existing doctor's Practitioner resource.

Request fields

- `doctorGcpFhirId` — required
- `doctorPrintName`, `medicalLicenseNumber`, `specialty`, `qualification` — required
- `gender` — `"Male" | "Female" | "Other"`
- `abdmProfessionalId` — optional

Error codes

- `VALIDATION_ERROR` (400)
- `DOCTOR_NOT_FOUND` (404) — `doctorGcpFhirId` does not exist in the FHIR store
- shared codes

7.5 `/get_doctor_opd_patients_by_date` — POST (auth)

Lists a doctor's OPD patients for a date.

Request fields

- `date` — required
- `doctorGcpId` — required

Error codes: shared codes only.

Note: `orgId` in the body (if present) is ignored — the organization is taken from the token.

7.6 `/get_doctor_ipd_patients_by_date` — POST (auth)

Lists a doctor's IPD (in-patient) patients.

Request fields

- `doctorGcpId` — required
- `date` — optional (no-op)

Error codes: shared codes only.

Note: `orgId` in the body (if present) is ignored.

7.7 `/patient_by_id` — POST (auth)

Fetches a single patient by MySQL id.

Request fields

- `patientId` — number (required)

Error codes

- `PATIENT_NOT_FOUND` (404)
- shared codes

7.8 `/patient_by_abhaAddress` — POST (auth)

Fetches a single patient by ABHA address.

Request fields

- `abhaAddress` — required

Error codes

- `PATIENT_NOT_FOUND` (404)
- shared codes

7.9 `/patient_register_abha_otp_init` — POST (auth)

Step 1 of ABHA registration: search ABHA address / request OTP.

Request fields

- `abhaAddress` — required
- `authMode` — `"MOBILE_OTP" | "AADHAAR_OTP" | "DONT_KNOW"`

Error codes

- `VALIDATION_ERROR` (400)
- `ABDM_GATEWAY_ERROR` (502) — e.g. auth mode not available for this ABHA
- shared codes

7.10 `/patient_register_abha_otp_confirm` — POST (auth)

Step 2 of ABHA registration: verify OTP and create the patient.

Request fields

- `transactionId` — required
- `otp` — required
- `abhaAddress` — required

Error codes

- `VALIDATION_ERROR` (400)
- `ABDM_GATEWAY_ERROR` (502) — OTP verification failed / no user returned
- shared codes

7.11 `/opd_patient` — POST (auth)

Creates an OPD encounter for a patient.

Request fields

- `date` — required (ISO date)
- `doctorDetails` — array of `{ doctorName, doctorGcpId }`, min 1
- patientNeeds

Error codes

- `INVALID_DOCTOR_GCP_FHIR_ID` (400) — a `doctorGcpId` is not a valid Practitioner
- `PATIENT_NOT_FOUND` (404)
- shared codes

7.12 `/admit_patient` — POST (auth)

Creates an IPD admission encounter.

Request fields

- `doa` — required (ISO date)
- `doctorDetails` — array of `{ doctorName, doctorGcpId }`, min 1
- patientNeeds

Error codes

- `INVALID_DOCTOR_GCP_FHIR_ID` (400)
- `PATIENT_NOT_FOUND` (404)
- shared codes

7.13 `/discharge_patient` — PUT (auth)

Discharges a patient (closes the encounter).

Request fields

- `encounterId` — required
- `dod` — required (ISO date)
- `doctorDetails` — optional
- patientNeeds

Error codes

- `INVALID_DOCTOR_GCP_FHIR_ID` (400)
- `PATIENT_NOT_FOUND` (404)
- `ENCOUNTER_NOT_FOUND` (404) — encounter id not found
- shared codes

7.14 `/discharge_summary` — POST (auth)

Creates a DischargeSummary composition and pushes it to ABDM.

Request fields

- `doctorDetails` — array of `{ doctorName, doctorGcpId }`, min 1
- `text`, `status`, `date` — required
- `compositionId` — optional
- patientNeeds

Error codes

- `INVALID_DOCTOR_GCP_FHIR_ID` (400)
- `PATIENT_NOT_FOUND` (404) · `ENCOUNTER_NOT_FOUND` (404)
- `ABDM_GATEWAY_ERROR` (502) · `LINK_TOKEN_MISSING` (502)
- shared codes

7.15 `/diagnostic_report` — POST (auth)

Creates a DiagnosticReport composition and pushes it to ABDM.

Request fields

- `performer` — array of `{ doctorName, doctorGcpId }`, min 1
- `testName` — required, non-empty
- `category` — required; one of `"Hematology" | "Biochemistry" | "Microbiology" | "Radiology" | "Others"`
- `text`, `status` — required, non-empty
- `requester`, `conclusion`, `compositionId`, `date` — optional
- patientNeeds

Error codes

- `INVALID_DOCTOR_GCP_FHIR_ID` (400) — validates `performer` **and** `requester`
- `PATIENT_NOT_FOUND` (404) · `ENCOUNTER_NOT_FOUND` (404)
- `ABDM_GATEWAY_ERROR` (502) · `LINK_TOKEN_MISSING` (502)
- shared codes

Notes: `date` is optional and defaults to the request timestamp when omitted.

7.16 `/op_consultation` — POST (auth)

Creates an OP Consultation composition and pushes it to ABDM.

Request fields

- `doctorDetails` — array of `{ doctorName, doctorGcpId }`, min 1
- `date`, `status` — required

- optional: `compositionId`, `chiefComplaints`, `medicalHistory`, `physicalExamination`, `medicines[]`, `opdProcedure`, `followUp`
- patientNeeds

Error codes

- `INVALID_DOCTOR_GCP_FHIR_ID` (400)
- `PATIENT_NOT_FOUND` (404) · `ENCOUNTER_NOT_FOUND` (404)
- `ABDM_GATEWAY_ERROR` (502) · `LINK_TOKEN_MISSING` (502)
- shared codes

7.17 `/procedure_ot_notes` — POST (auth)

Creates a procedure/OT-notes composition and pushes it to ABDM.

Request fields

- `doctorDetails` — array of `{ doctorName, doctorGcpId }`, min 1
- `date`, `status` — required
- optional: `compositionId`, `opdProcedure`, `followUp`
- patientNeeds

Error codes

- `INVALID_DOCTOR_GCP_FHIR_ID` (400)
- `PATIENT_NOT_FOUND` (404) · `ENCOUNTER_NOT_FOUND` (404)
- `ABDM_GATEWAY_ERROR` (502) · `LINK_TOKEN_MISSING` (502)
- shared codes

7.18 `/health_document` — POST (multipart/form-data, auth)

Uploads a scanned health document (images) and creates a HealthDocumentRecord composition.

Request fields (form fields)

- `doctorDetails` — **JSON string** (required), e.g. `"[{\"doctorName\":\"Dr Umesh\",\"doctorGcpId\":\"...\"}]"`
- `text`, `status` — required
- `date`, `compositionId` — optional
- `abhaAddress` or `patientId` — at least one (patientNeeds)

Files

- `page1` — required (image, ≤ 2 MB)
- `page2`, `page3`, `page4` — optional

Error codes

- `INVALID_REQUEST_METHOD` (400) — non-POST
- `DOCTOR_DETAILS_REQUIRED` (400) — `doctorDetails` missing
- `VALIDATION_ERROR` (400) — invalid form fields
- `INVALID_DOCTOR_GCP_FHIR_ID` (400)
- `PATIENT_NOT_FOUND` (404) · `ENCOUNTER_NOT_FOUND` (404)
- `ABDM_GATEWAY_ERROR` (502) · `LINK_TOKEN_MISSING` (502)
- shared codes

---

8. Not covered here

- `/uhi` and `/uhi/whatsapp` follow the **UHI protocol** (search/init/confirm/cancel/status), which has its own acknowledgement and error format — documented separately, not via this envelope.

---

9. Quick reference — which code for which mistake

You did this	You'll see
Missing/invalid token	`401 UNAUTHORIZED`
Expired subscription	`403 SUBSCRIPTION_EXPIRED`
Bad field / wrong enum / missing required field	`400 VALIDATION_ERROR` (+ `details`)
Wrong `doctorGcpId`	`400 INVALID_DOCTOR_GCP_FHIR_ID`
Unknown `patientId` / `abhaAddress`	`404 PATIENT_NOT_FOUND`
Unknown `encounterId`	`404 ENCOUNTER_NOT_FOUND`
Duplicate email on create_doctor	`409 EMAIL_ALREADY_EXISTS`
ABDM OTP/search/verify failure	`502 ABDM_GATEWAY_ERROR`
ABDM link token missing after callback	`502 LINK_TOKEN_MISSING`
Anything else unexpected	`500 INTERNAL_ERROR` / `500 DB_ERROR`